Compare → CMP

CMP Ax, Bx → Sour

Ax
- Bx

Ax → Destination

CMP → works by Subtracting one no from another. But doesn't Store the result. Instead it updates the cpu flags. OF, CF, ZF, SF

2's comp → neg no.

2 = 0010

1101

+1

(1110) → -2

Dest = Src

(ZF = 1)

1011
- 1011
0000

Dest < Src

unsigned positive only

(CF = 1)

10110 6
- 1000 8
1110

Dest <= Src → ZF = 1 OR CF = 1

(unsigned) positive (equal) (<)

Unsigned dest >= Unsigned Src

```
  0'x'b'00   8
- 0110       6
----------
  0010       (2)
```

(CF = 0)

UDest > USrc

ZF = 0 & CF = 0

↓

not equal

---

Signed Destination < Signed Source  SF ≠ OF

```
  '0'10'
  '0'x'01     5
- 0110        (6)
----------
  1111
```

(OF = 0) (SF = 1)

↳

there exist Both Borrow in and Borrow out hence OF = 0

```
  (x)(x)
   1110      -2
- 0010       +2
----------
  1100       -4
```

(SF = 1)

there exist no Borrow in \Borrow out hence OF = 0

SDest $\leq$ SSrc  ZF=1 or SF $\neq$ OF

SDest $\geq$ SSrc  SF = OF

```
  1110
- 1110
------
  0000
```

OF = 0  SF = 0

↳ no carry in / carry out

```
  11X0      -2
- 1101      -3
------
  0001
```

OF = 0  SF = 0

```
  y010 10     2
- 1110        -2
------
  0100
```

OF = 0  SF = 0

9) SDest > SSrc

ZF = 0 and SF = OF

---

Types of jumps

Below / above → talking about positive nos / unsigned no.

greater / less → talking about Signed nos

if we want to define data at top : use JMP. this is unconditional

```
Jmp Start
num1: dw 0, 10, 15
total : dw 0
Start : mov ax, 0
        :
        int 21h
```

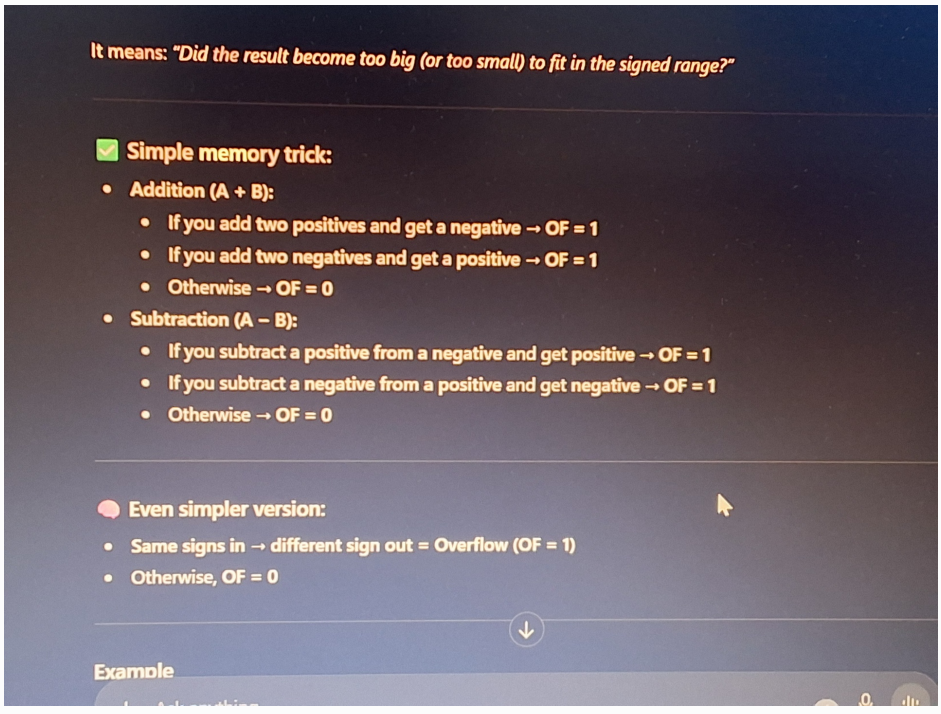
goes over here

Here JMP Start is stored first in memory and data is stored right after it.

The cpu jumps so that it doesn't accidentally execute data.

memory order → JMP → data → code

execution order → JMP → code




Target = IP + Displacement
(opcode)

Displacement = target − Next(IP)

Short Jmp

opcode
EB F2
1 byte
address (Displacement)

near Jmp

address (displacement)
E9 1600
opcode

For Jump

Base
EB IDEA 3901
opcode offset

EA ID

01 39

(no relative addressing)

Move Base address to CS

Move offset to IP

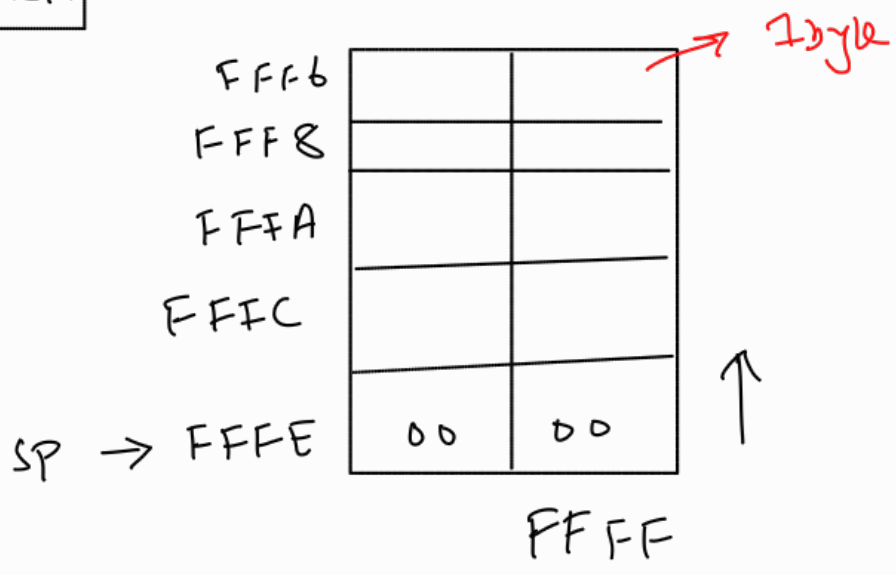
conditional jumps are are short.

unconditional jumps can be near | short | far

In far jump the complete segment changes

---

Stack




Decrementing stack

Utilizes mem from higher → lower address

e.g

case → Near Jump

---

to put value in Stack: Push

Push ax → Source means Stack is destination

IP 0119

→ FFFC | 2E | 01

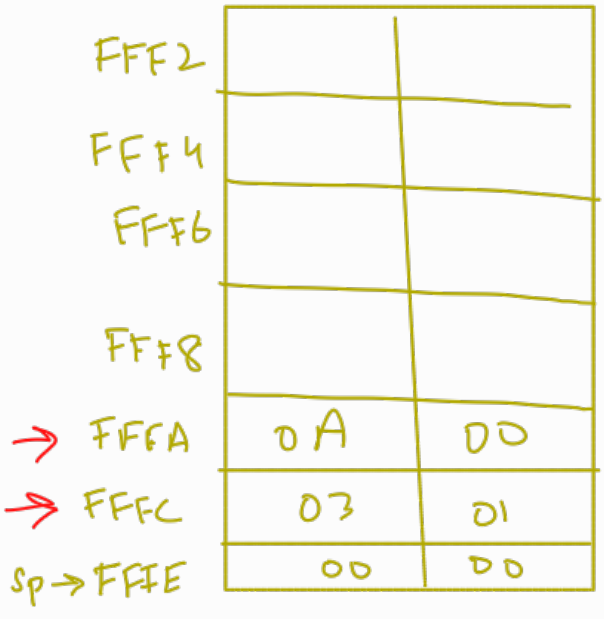
push, pop → doesn't let the function change the values of registers so that main body uses them as it is.

pop ax — whereas stack is source

destination

---

ax | 10

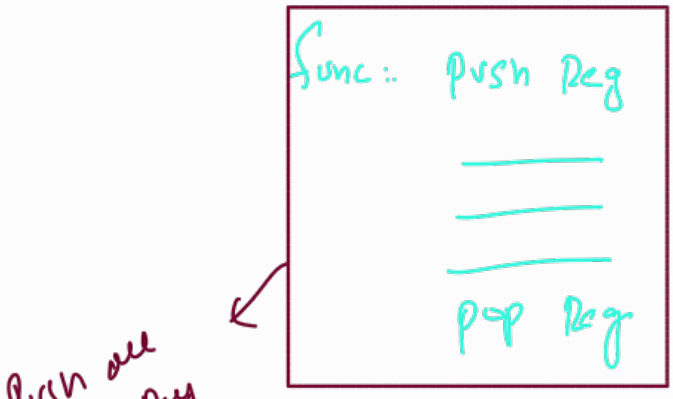



Loop keyword ↻ cx = cx - 1 ; Jump if cx != 0

case
Decrement Sp by 2 Sp = Sp-2 ←
copy IP value on stack ←
Relative addressing → place in IP

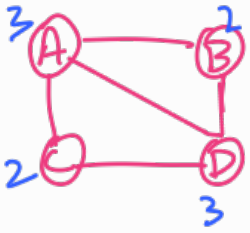
Ret

1) copy top of the Stack to IP
2) Increment Sp by 2

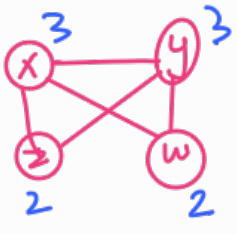



Push all the Reg that are needed inside the function

If we bring any parameters from outside in the function then Return 2x n → no of values pushed

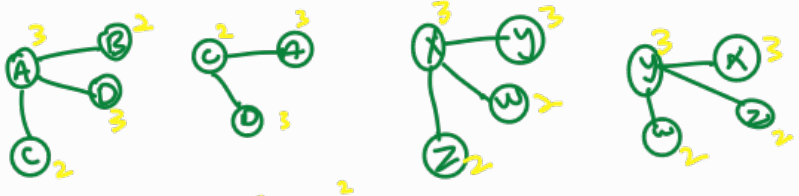



3,3,2,2
(g1)

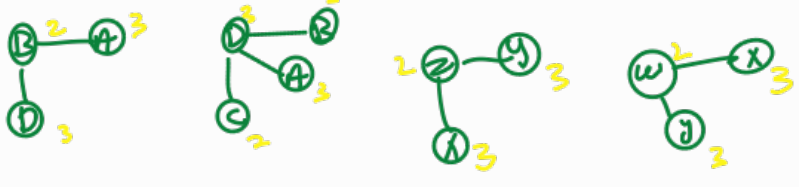



3,3,2,2
(g2)

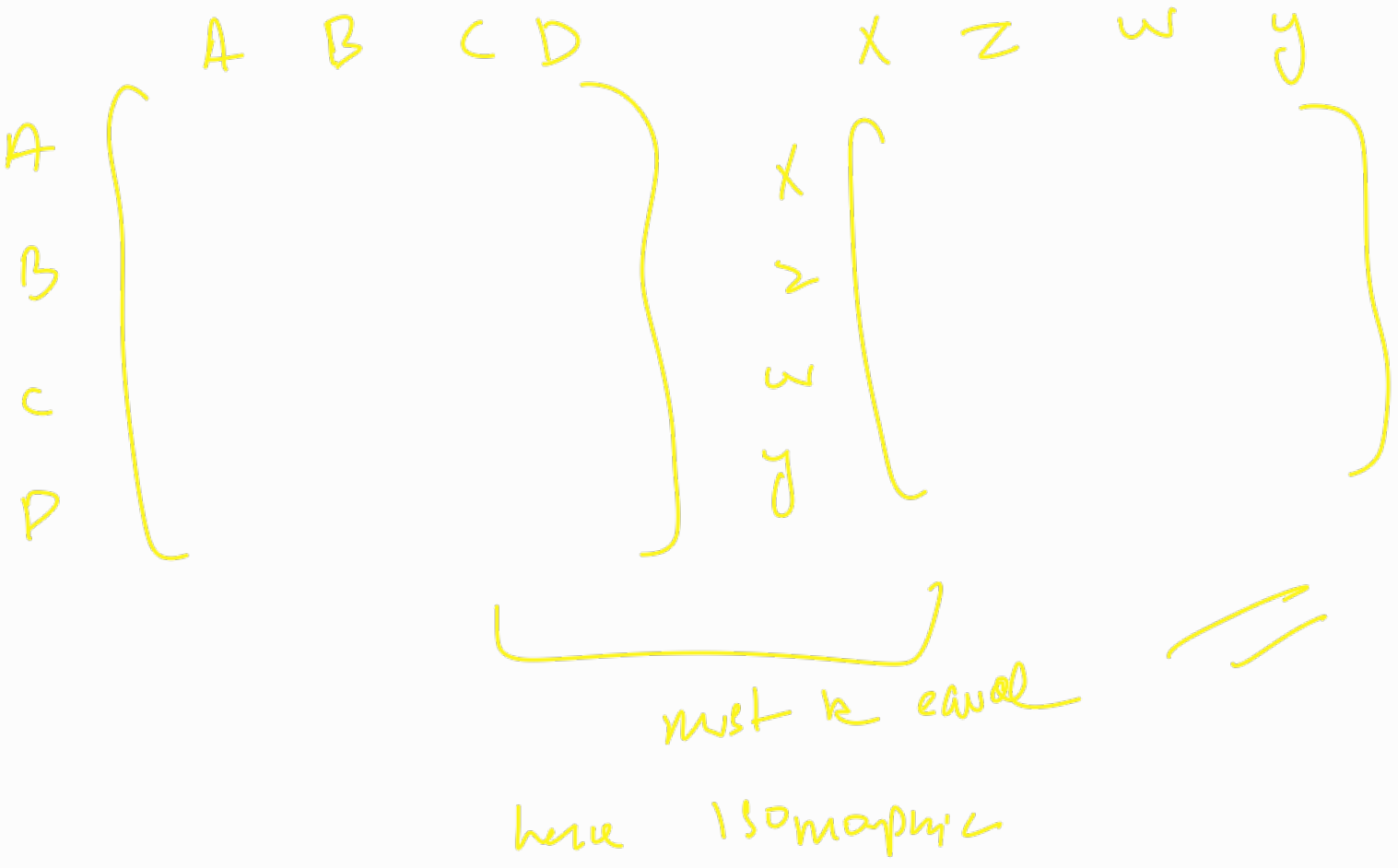
| edges | |
|---|---|
| A-B | x-z |
| A-D | x-y |
| A-C | x-w |
| B-D | z-y |
| C-D | w-y |



A=x
B=z
C=w
D=y



(g1) (g2)




here isomorphic

[cs: bx + di]

phy: seg x16 + offset

Bx = 00ff
CS = 1111
~~DS = 2222~~
~~SS = 2520~~

~~IP = 1232~~
SP = 1100
DI: 0020

(1111 x16) + ( 00ff + 0020)

11110 + 00020 → 11130h

effective address → offset
physical address = (seg x16) + offset

calculator hexadecimal letter → Shift + Letter

XCHG ax, bx → Swap values of these

add word [sum], [bx+si] memory to memory is invalid

mov ax, [bx+si]
add [sum], ax

* ASCII → 0x2A

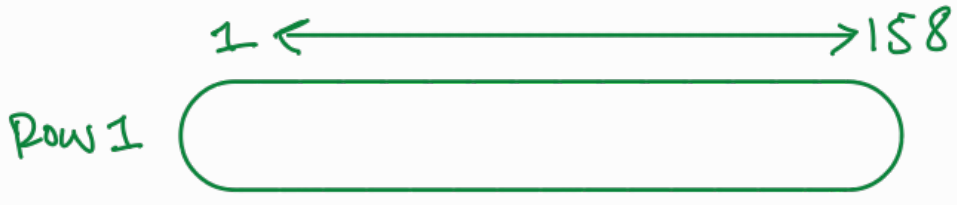
Mov [es:di],

num2 | FF | 00 |

num2 : dw 00FF

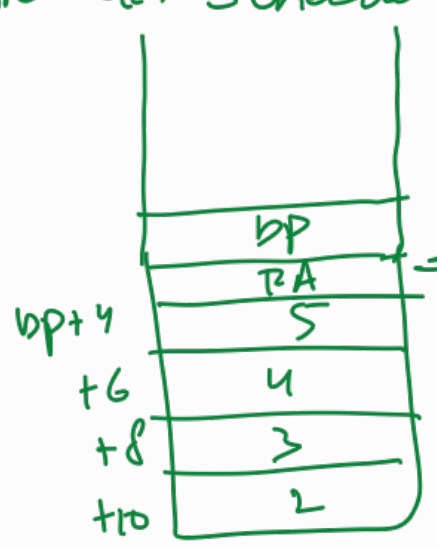
[num2] → FF

[num2+1] → 00




at 158 ASCII is stored
and at 159 the attribute
is stored and then at
160 Row 2 Seconds

Int 08h → timer calls Schedular



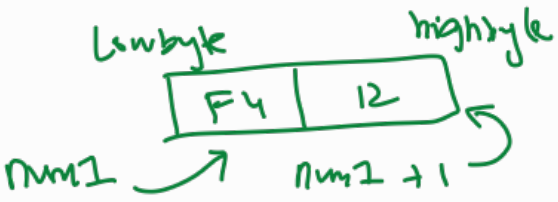

→ call

ax:- bp
bx:- RA

(bx:- RA - 2)
ax:-(bp - 3)

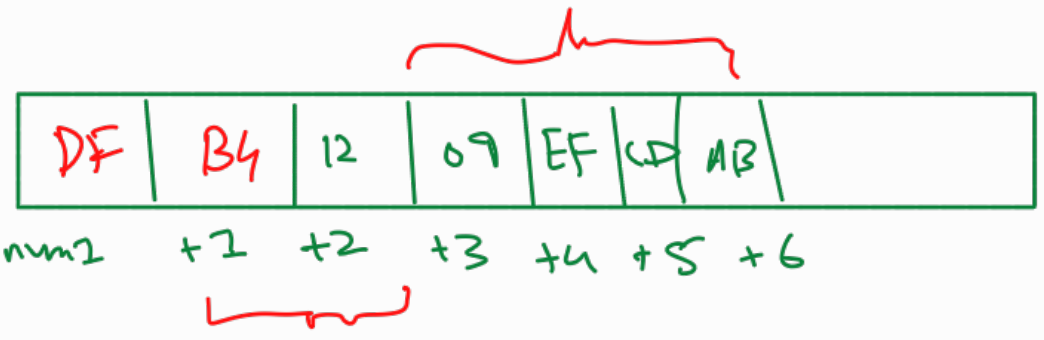
([RA-2 + bp-3])

num1 : dw 0x12F4




0x ABCDEF19

num1 : db 0xA



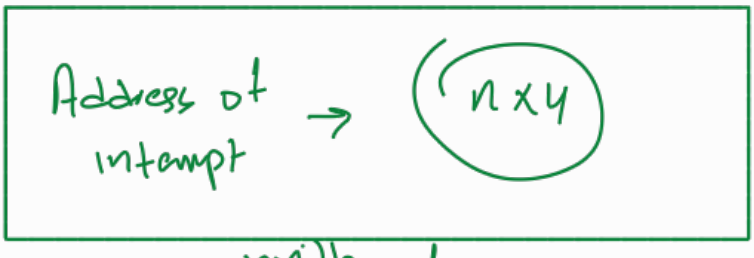

mov ax, [num1 + 5]

ax → ABCD

if dx <= cx , ax=1 else ax=2

less than equal

```
cmp dx, cx
JNGE L1
mov ax,2
Jmp done
L1
    mov ax,1
```




write at
0000: offset




| key | Scan code |
|---|---|
| 1 | 02h |
| 2 | 03h |
| ⋮ | |
| 9 | |
| 0 | 0Bh |

Push ds
pop es

ES ← points to data
Seg where
data is stored
instead of
Video memory

CS:- 2000h

DS:- 3000h

SS:- 4000h

BX:- 0100h

SI:- 0020h

BP:- 0010